

EXPERIMENT: Write a Prolog Program to implement Best First Search algorithm

Aim

To implement a Prolog program for Best First Search (BFS) to find a path from a starting node to a goal node using heuristic values.

Objectives

- To understand the concept of Best First Search.
- To represent a graph using Prolog facts.
- To use heuristic values to select the most promising node.
- To find a path from a start node to a goal node.

Algorithm

1. Start with the initial node.
2. Put the initial node in the OPEN list.
3. Select the node with the lowest heuristic value.
4. If the selected node is the goal, stop.
5. Otherwise, find all its neighboring nodes.
6. Add the unvisited neighboring nodes to the OPEN list.
7. Again select the node with the lowest heuristic value.
8. Repeat until the goal is reached.
9. Display the path.

CODE

```
graph = {  
    'A': ['B', 'C'],  
    'B': ['D', 'E'],  
    'C': ['F', 'G'],  
    'D': ['H'],  
    'E': ['H'],  
    'F': ['H'],
```

```

    'G': ['H'],
    'H': []
}
heuristic = {
    'A': 10,
    'B': 6,
    'C': 4,
    'D': 3,
    'E': 2,
    'F': 2,
    'G': 1,
    'H': 0
}
def best_first_search(start, goal):
    open_list = [(heuristic[start], start)]
    visited = set()
    parent = {start: None}
    while open_list:
        open_list.sort()
        h, current = open_list.pop(0)
        if current in visited:
            continue
        visited.add(current)
        print("Visiting:", current)
        if current == goal:
            path = []
            while current is not None:

```

```

        path.append(current)
        current = parent[current]

    path.reverse()
    return path
for neighbor in graph[current]:
    if neighbor not in visited:
        if neighbor not in parent:
            parent[neighbor] = current
            open_list.append((heuristic[neighbor], neighbor))
return None
start = input("Enter start node: ").upper()
goal = input("Enter goal node: ").upper()
path = best_first_search(start, goal)
if path:
    print("\nBest First Search Path:")
    print(" -> ".join(path))
else:
    print("\nPath not found.")

```

OUTPUT

```
IDLE Shell 3.12.5
File Edit Shell Debug Options Window Help
Python 3.12.5 (tags/v3.12.5:ff3bc82, Aug 6 2024, 20:45:27) [MSC v.1940 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\shaik arshad basha\AppData\Local\Programs\Python\Python312\python.exe
Enter start node: A
Enter goal node: H
Visiting: A
Visiting: C
Visiting: G
Visiting: H

Best First Search Path:
A -> C -> G -> H
>>> |
```

Ln: 14 Col: 0

Result

The Best First Search algorithm was successfully implemented in Prolog. The program uses heuristic values to select the most promising node and finds a path from the starting node to the goal node.

Conclusion

Best First Search is an informed search technique that uses heuristic information to guide the search toward the goal. The Prolog program demonstrates how graphs, heuristic values, recursion, and list processing can be combined to implement this algorithm.